El motor de inferencia en C

Guía para escribir la decisión H10 del recomendador de CPIA

Arnau Montull para Uriel Martínez

16 de octubre de 2026

Resumen

Esta guía explica qué tiene que hacer el motor de inferencia en C del recomendador (H10), en qué formato le llegarán los datos desde Python, qué funciones escribir (con firma y pseudocódigo), cómo paralelizarlo con OpenMP, cómo comprobar que da lo mismo que numpy y cómo medirlo para las entregas E2/E3. *No* es la implementación: la escribes tú. Todo está en C puro (nada de C++), compila con el gcc 13.1 de Simply Fortran y se llama desde Python con `ctypes`. Las cifras salen de la submuestra de desarrollo (`reports/pipeline_dev.md`): $N = 23\,335$ películas, $d = 128$. El modelo PyTorch aún no existe, así que el formato de exportación (sección 2) es una **propuesta provisional**.

Índice

1. Qué hace el motor y qué no **3**
1.1. El reparto . . . 3
1.2. El flujo completo . . . 3
1.3. Cifras de partida (dev) . . . 4

2. Formato de los datos que exportará Python **4**
2.1. Reglas generales . . . 4
2.2. Archivos . . . 4
2.3. `meta.txt` . . . 5
2.4. Leer con `fread` . . . 5

3. Los módulos en C **5**
3.1. Estructuras . . . 5
3.2. `cargar_modelo` . . . 6
3.3. `media_bolsa` . . . 6
3.4. `mlp_usuario` . . . 6
3.5. `normalizar_l2` . . . 7
3.6. `scores` . . . 7
3.7. `aplicar_mascara` . . . 7
3.8. `top_k` con un min-heap . . . 7
3.9. `mmr` . . . 7

4. OpenMP y SIMD **8**
4.1. Qué paralelizar . . . 8
4.2. SIMD sin intrínsecos . . . 9
4.3. Compilar . . . 9

5. Validación **9**
5.1. Referencia numpy . . . 9

5.2. Qué comparar 10
5.3. Casos límite (un test cada uno) 10

6. Benchmarks **10**
6.1. Orden de magnitud antes de medir 10
6.2. Qué medir 11
6.3. Llamar a la `.dll` desde Python 11
6.4. Comparación con `hnswlib` 12
6.5. Para la memoria (E2/E3) 12

7. Estructura y orden de trabajo **12**
7.1. Carpetas 12
7.2. Pasos (cada uno se comprueba antes de seguir) 12

1. Qué hace el motor y qué no

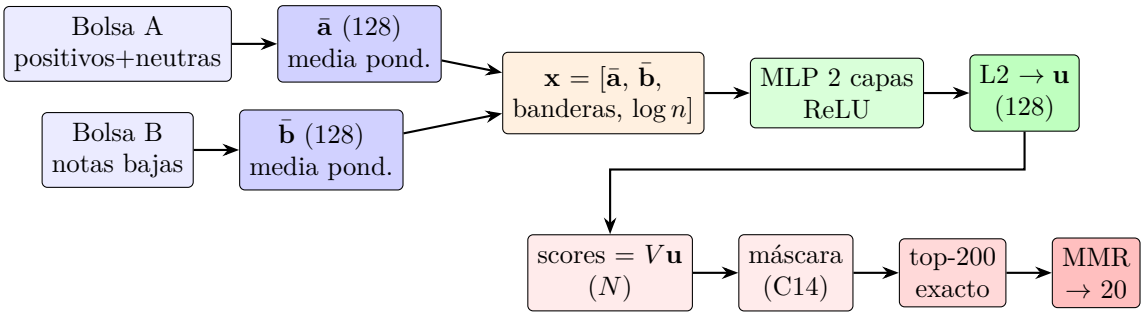
1.1. El reparto

- **Entrenamiento**: en PyTorch (H8, en Colab o CPU). Aún no existe. Produce los vectores de película $\mathbf{v}_i$ y los pesos del MLP de la torre de usuario.
- **Motor en C (esto)**: solo *inferencia*. Entrada: el historial de un usuario. Salida: 20 películas. No entrena, no calcula gradientes, no lee texto ni sinopsis: los $\mathbf{v}_i$ ya vienen calculados.
- **HNSW**: no lo escribes en C. Con 23 335 películas la búsqueda exacta ya cumple los < 5 ms de la E1 (H9). HNSW se mide aparte con `hnswlib` en Python, como comparación (también tuyo, sección 6.4).

Lo prometido en la E1

200 candidatos en < 5 ms, reordenación MMR hasta 20, OpenMP y SIMD, y benchmarks en CPU. La E1 hablaba de 84 000 películas; tras la limpieza (mín. 20 ratings) son 23 335, así que todo es 3,6× más barato de lo que escribimos allí.

1.2. El flujo completo



En fórmulas, para un usuario con bolsa A (índices j, pesos w_j) y bolsa B (índices k, pesos w_k):

$$\bar{\mathbf{a}} = \frac{\sum_j w_j \mathbf{v}_j}{\sum_j w_j}, \qquad \bar{\mathbf{b}} = \frac{\sum_k w_k \mathbf{v}_k}{\sum_k w_k} \quad (\text{o } \mathbf{0} \text{ si la bolsa está vacía}),$$

$$\mathbf{x} = [\bar{\mathbf{a}},\ \bar{\mathbf{b}},\ f_A,\ f_B,\ \log(1+n)] \in \mathbb{R}^{d_0},$$

$$\mathbf{h} = \mathrm{ReLU}(W_1\mathbf{x} + \mathbf{b}_1), \qquad \mathbf{z} = W_2\mathbf{h} + \mathbf{b}_2, \qquad \mathbf{u} = \mathbf{z}/\|\mathbf{z}\|_2,$$

$$s_i = \langle \mathbf{v}_i, \mathbf{u} \rangle \ (i = 1..N), \qquad s_i = -\infty \text{ si } i \text{ ya está valorada},$$

y luego top-200 de s y MMR sobre esos 200. Las bolsas son las de F4: en A entran positivos (peso C4×C9) y neutras (peso 0,25×C9); en B las notas bajas (peso C9). Los $\mathbf{v}_j$ de las bolsas son los mismos de la torre de ítem (F6).

Detalles que dependen del modelo

Qué banderas hay exactamente (f_A, f_B = «bolsa vacía»), si n es el tamaño total o de cada bolsa, y si hay ReLU tras la segunda capa, lo fijará el modelo PyTorch. Por eso el motor *no* lo debe codificar a fuego: lo lee del `meta.txt` (sección 2). Aquí asumimos $d_0 = 128 + 128 + 2 + 1 = 259$.

1.3. Cifras de partida (dev)

	p50	p99 (máx.)
Tamaño de la bolsa A (train+val)	57	997 (4 237)
Tamaño de la bolsa B (train+val)	8	184 (944)
Usuarios con B vacía (train+val)		951 de 20 085

Las medias de las bolsas son baratas ($\sim 60 \times 128$ sumas); lo caro es $V\mathbf{u}$: $23\,335 \times 128 \approx 3$ M multiplicaciones-suma.

2. Formato de los datos que exportará Python

Provisional

Todo este apartado es una **propuesta de contrato**. Se cerrará cuando exista el modelo. Mientras tanto, genera estos archivos con datos *aleatorios* desde Python (sección 5) y programa contra ellos.

2.1. Reglas generales

- Binarios crudos, sin cabecera, **little-endian** (lo normal en x86) y **row-major** (orden C, el de numpy por defecto). Se escriben con `arr.astype('<f4').tofile(ruta)`.
- Tipos: `float32` (`.f32`) para vectores y pesos; `int32` (`.i32`) para índices de película; `int64` (`.i64`) para los `indptr` de CSR (con el conjunto completo pueden pasar de 2^{31}).
- Un `meta.json` (para humanos y Python) y un `meta.txt` con lo mismo en `clave = valor`, para no tener que parsear JSON en C.

2.2. Archivos

Archivo	Forma, tipo	Contenido
`V.f32`	$[N \times d]$ float32	$\mathbf{v}_i$ ya normalizados (norma 1, E7). Fila i = película de índice interno i.
`W1.f32`, `b1.f32`	$[h_1 \times d_0]$, $[h_1]$	Primera capa. Ojo: `nn.Linear` guarda `weight` como [salida × entrada]; se exporta tal cual.
`W2.f32`, `b2.f32`	$[d \times h_1]$, $[d]$	Segunda capa.
`movie_ids.i32`	$[N]$	`movieId` de MovieLens de cada índice interno (solo para mostrar).
`usuario_A_idx.i32`, `usuario_A_w.f32`	$[n_A]$	Bolsa A del usuario de ejemplo.
`usuario_B_idx.i32`, `usuario_B_w.f32`	$[n_B]$	Bolsa B.
`usuario_excl.i32`	$[n_E]$	Lo ya valorado (máscara C14/F8).
`lote_*` (opcional)	CSR	Varios usuarios: `indptr.i64` $[B+1]$, `idx.i32`, `w.f32`, para A, B y excl.
`ref_u.f32`, `ref_scores.f32`, `ref_top.i32`		Salidas de referencia de numpy para validar.

2.3. meta.txt

```
# contrato de exportación v0 (provisional)
version = 0
N = 23335
d = 128
d0 = 259           # dimension de la entrada del MLP
h1 = 256           # capa oculta (provisional)
relu_final = 0     # 1 si hay ReLU tras la segunda capa
orden_entrada = media_A,media_B,vacia_A,vacia_B,log1p_n
n_para_log = A+B   # que n entra en log(1+n)
normalizado_V = 1
lambda_mmr = 0.7
```

El orden de la entrada es lo más delicado: si C concatena en otro orden que PyTorch, todo «funciona» y da basura. Por eso va escrito en el meta y el motor debe comprobar que coincide con el que espera (y abortar si no).

2.4. Leer con `fread`

```c
/* Lee exactamente n floats de 'ruta' en un buffer nuevo; NULL si falla. */
float *leer_f32(const char *ruta, size_t n) {
    FILE *f = fopen(ruta, "rb");          /* "rb": binario, imprescindible en Windows */
    if (!f) return NULL;
    float *p = aligned_alloc_64(n * sizeof(float));
    size_t leidos = fread(p, sizeof(float), n, f);
    /* comprobar que no sobra nada: el tamaño del archivo debe ser n*4 */
    int extra = fgetc(f);
    fclose(f);
    if (leidos != n || extra != EOF) { liberar_64(p); return NULL; }
    return p;
}
```

Y `meta.txt` se lee línea a línea con `fgets` + `sscanf(linea, "%63[^= ] =%255[^#\n]", clave, valor)`, ignorando las que empiezan por `#`.

Windows

`aligned_alloc` no existe en el runtime de MinGW: usa `_aligned_malloc(n, 64)` / `_aligned_free` (en `<malloc.h>`). Escóndelo en dos funciones propias (`aligned_alloc_64`, `liberar_64`) para no ensuciar el resto. Y abre siempre con `"rb"`: en modo texto Windows traduce bytes `0x1A` y `\r\n`.

3. Los módulos en C

3.1. Estructuras

```c
typedef struct {
    int N, d, d0, h1, relu_final;
    float lambda_mmr;
    float *V;            /* [N*d], normalizada, alineada a 64 */
    float *W1, *b1;      /* [h1*d0], [h1] */
    float *W2, *b2;      /* [d*h1],  [d]  */
    int32_t *movie_ids;  /* [N] */
} Modelo;

typedef struct {         /* una bolsa: vista sobre un trozo de CSR */
```

```
    const int32_t *idx; const float *w; int n;
} Bolsa;
```

3.2. cargar_modelo

```
int  cargar_modelo(const char *carpeta, Modelo *m);   /* 0 = ok; <0 = código de error */
void liberar_modelo(Modelo *m);
```

1. Leer `meta.txt`; comprobar `version`, `orden_entrada` y que $d_0 = 2d + 3$.
2. `leer_f32` de `V`, `W1`, `b1`, `W2`, `b2` con los tamaños del meta (si el tamaño del archivo no cuadra, error: así se detecta un meta desactualizado).
3. Comprobación barata: la norma de unas cuantas filas de V vale 1 ± 10^{-3}.

3.3. media_bolsa

Es la traducción de `media_bolsa` de `src/preprocesamiento/historial.py` para un solo usuario (sin la parte de `excluir_item`, que solo sirve al entrenar).

```
/* out[d] = sum w_j V[idx_j] / sum w_j. Devuelve 1 si la bolsa está vacía (out = 0). */
int media_bolsa(const Modelo *m, Bolsa b, float *restrict out);
```

```
acum[0..d) = 0  (en double, como numpy)
W = 0
para j en 0..n-1:
    fila = V + idx[j]*d
    para c en 0..d-1: acum[c] += w[j] * fila[c]
    W += w[j]
si W <= 1e-9:  out = 0;  devolver 1        # bandera "vacía"
out[c] = acum[c] / W;  devolver 0
```

Bolsa vacía

Hay 951 usuarios con B vacía en dev, y un usuario de Letterboxd recién llegado puede tener todo vacío. Nunca dividas por cero: vector **0** y bandera a 1. El umbral 10^{-9} es el mismo de numpy (cubre pesos que suman 0).

3.4. mlp_usuario

```
/* Construye x, aplica las dos capas y deja z[d] (sin normalizar). tmp: buffer de h1
    floats. */
void mlp_usuario(const Modelo *m, Bolsa A, Bolsa B, float *restrict z, float *restrict
    tmp);
```

```
x[0..d)    = media_bolsa(A) ;  fA = bandera
x[d..2d)   = media_bolsa(B) ;  fB = bandera
x[2d] = fA;  x[2d+1] = fB;  x[2d+2] = log1p(nA + nB)     # según orden_entrada del meta
para o en 0..h1-1:  tmp[o] = max(0, b1[o] + sum_c W1[o*d0 + c] * x[c])
para o en 0..d-1:   z[o]   = b2[o] + sum_c W2[o*h1 + c] * tmp[c]
si relu_final: z = max(0, z)
```

Cada capa es un producto matriz-vector: escribe una función `matvec(W, x, filas, cols, out)` y úsala aquí y en `scores`.

3.5. `normalizar_l2`

```c
void normalizar_l2(float *restrict v, int d);   /* v /= ||v||; si ||v|| < 1e-12, deja v
    igual */
```

PyTorch (`F.normalize`) usa $\max(\|v\|, \varepsilon)$ con $\varepsilon = 10^{-12}$; imita eso.

3.6. `scores`

```c
/* s[i] = <V[i], u> para i = 0..N-1 */
void scores(const float *restrict V, const float *restrict u, int N, int d, float *
    restrict s);
```

Dos bucles: el externo sobre i (paralelizable), el interno sobre c (vectorizable). Es el 90 % del tiempo.

3.7. `aplicar_mascara`

```c
void aplicar_mascara(float *s, const int32_t *excl, int n_excl);  /* s[excl[j]] = -
    INFINITY */
```

Lo excluido es todo lo valorado en el historial (C14; en test, train+val por F8). Usa `-INFINITY` de `<math.h>`, igual que `top_k` de `src/evaluacion/metricas.py`, y luego en `top_k` descarta los $-\infty$ para no devolverlos nunca.

3.8. `top_k` con un min-heap

```c
typedef struct { float s; int32_t i; } Par;
/* Escribe en out[] los k mejores (score descendente). Devuelve cuántos hay (<= k). */
int top_k(const float *s, int N, int k, Par *out);
```

Por qué un heap

Ordenar los 23 335 scores es $O(N \log N)$ y mueve mucha memoria. Basta con mantener los $k = 200$ mejores vistos hasta ahora en un **min-heap** (la raíz es el *peor* de los 200): cada score nuevo se compara solo con la raíz; casi siempre pierde y cuesta una comparación. Coste $O(N \log k)$ en el peor caso y $\approx O(N)$ en la práctica.

```
cnt = 0
para i en 0..N-1:
    si s[i] == -inf: continuar
    si cnt < k:  heap[cnt++] = (s[i], i);  subir(heap, cnt-1)
    si no, si mejor((s[i],i), heap[0]):  heap[0] = (s[i], i);  hundir(heap, cnt, 0)
ordenar heap[0..cnt) de mayor a menor (qsort, son solo 200)
devolver cnt
```

`mejor(a, b)` es «`a.s > b.s`, y si empatan, `a.i < b.i`». Con este desempate el resultado es determinista y coincide con el criterio del índice menor de numpy dentro del trozo. Si quedan menos de k películas sin excluir, se devuelven las que haya (`cnt < k`): el que llama debe usar el valor devuelto, no k.

3.9. `mmr`

MMR (*Maximal Marginal Relevance*) elige 20 de los 200 candidatos de forma voraz, equilibrando relevancia y diversidad. En cada paso, entre los candidatos c aún no elegidos, toma

$$c^* = \arg\max_{c \notin S} \Big[\lambda\, s_c \;-\; (1-\lambda)\, \max_{e \in S} \langle \mathbf{v}_c, \mathbf{v}_e \rangle \Big],$$

donde $s_c = \langle \mathbf{u}, \mathbf{v}_c \rangle$ y S es lo ya elegido (en el primer paso el segundo término es 0, así que la primera es la más relevante). Como V está normalizada, la similitud coseno es solo el producto escalar (E7). $\lambda = 0{,}7$ por defecto, como parámetro: $\lambda = 1$ reproduce el top-20 puro.

Tres candidatos y elegir 2

Con $\lambda = 0{,}7$. Scores: $s_A = 0{,}90$, $s_B = 0{,}88$, $s_C = 0{,}70$. Similitudes: $\langle A, B \rangle = 0{,}95$ (dos entregas de la misma saga), $\langle A, C \rangle = 0{,}10$. Paso 1: gana A ($0{,}7 \cdot 0{,}90 = 0{,}63$). Paso 2: B vale $0{,}7 \cdot 0{,}88 - 0{,}3 \cdot 0{,}95 = 0{,}331$; C vale $0{,}7 \cdot 0{,}70 - 0{,}3 \cdot 0{,}10 = 0{,}460$. Gana C: se sacrifica algo de relevancia para no repetir saga.

```
/* cand[nc] viene de top_k. Escribe en out[] hasta k_final índices de película. */
int mmr(const Modelo *m, const Par *cand, int nc, int k_final, float lambda, int32_t *
    out);
```

```
maxsim[0..nc) = -inf ;  usado[0..nc) = 0
para paso en 0..min(k_final,nc)-1:
    mejor = -1
    para c en 0..nc-1 no usado:
        val = lambda*cand[c].s - (1-lambda)*(paso==0 ? 0 : maxsim[c])
        si val > val_mejor: mejor = c
    usado[mejor] = 1;  out[paso] = cand[mejor].i
    para c no usado:  maxsim[c] = max(maxsim[c], <V[cand[c].i], V[cand[mejor].i]>)
```

El truco es actualizar `maxsim` incrementalmente con el último elegido: coste $O(k_{\text{final}} \cdot nc \cdot d) = 20 \cdot 200 \cdot 128 \approx 0{,}5$ M operaciones, nada.

4. OpenMP y SIMD

4.1. Qué paralelizar

- `scores` sobre las N películas:

```
#pragma omp parallel for schedule(static)
for (int i = 0; i < N; i++) {
    const float *restrict fila = V + (size_t)i * d;
    float acc = 0.0f;
    #pragma omp simd reduction(+:acc)
    for (int c = 0; c < d; c++) acc += fila[c] * u[c];
    s[i] = acc;
}
```

- **top-k por hilos**: cada hilo hace su propio heap de k sobre su trozo de $[0, N)$, y al final se fusionan los T heaps ($T \cdot k$ pares, p. ej. $12 \cdot 200$) con un último `top_k` secuencial. Aún mejor: fusionar `scores`+máscara+heap en el mismo bucle por hilo, para no volver a leer s.
- **Lote de usuarios** (benchmarks, evaluación): paraleliza *por usuario* (`#pragma omp parallel for schedule(dynamic)`, porque las bolsas tienen tamaños muy distintos) y dentro de cada usuario todo secuencial. Cada hilo necesita sus propios buffers (`s`, `tmp`, heap): resérvalos dentro de la región paralela o un bloque por hilo con `omp_get_thread_num()`.

Qué no paralelizar

MMR ($nc = 200$), el MLP (~ 100 k operaciones) y las medias de bolsa son demasiado pequeños: crear el equipo de hilos cuesta del orden de microsegundos y se come la ganancia. Y nunca anides dos `parallel for` (por usuario y por película a la vez).

4.2. SIMD sin intrínsecos

- Compila con `-O3 -march=native`: gcc vectoriza solo el bucle interno con AVX2 (el i7-1355U no tiene AVX-512). Compruébalo con `-fopt-info-vec-optimized`.
- `restrict` en los punteros le promete a gcc que no se solapan; sin eso a veces no vectoriza.
- Una suma en float no es asociativa: gcc no la reordena (y por tanto no la vectoriza) sin `#pragma omp simd reduction` o `-ffast-math`. Usa el pragma (necesita `-fopenmp` o `-fopenmp-simd`).
- Alineación: V reservada a 64 bytes; como $d = 128$ floats $= 512$ bytes, todas las filas quedan alineadas. Puedes avisar a gcc con `__builtin_assume_aligned(V, 64)`.
- Intrínsecos AVX2 a mano (`_mm256_fmadd_ps`) solo si al medir el automático se queda corto; es un buen experimento opcional para la memoria.

4.3. Compilar

Con el gcc 13.1 de Simply Fortran (no hace falta instalar nada):

```
set PATH=C:\Program Files (x86)\Simply Fortran 3\mingw-w64\bin;%PATH%
gcc -O3 -march=native -fopenmp -Wall -Wextra -Iinclude src\*.c tests\test_motor.c -o
    build\test_motor.exe -lm
gcc -O3 -march=native -fopenmp -shared -Iinclude src\*.c -o build\motor.dll -lm
```

Para la `.dll`, marca las funciones exportadas con `__declspec(dllexport)` (una macro `MOTOR_API` en `motor.h`). La `.dll` necesitará en tiempo de ejecución `libgomp-1.dll` y compañía, que están en esa misma carpeta `bin`: o la añades al `PATH`, o en Python haces `os.add_dll_directory(r"C:\...\bin")` antes de cargarla. El número de hilos se controla con `OMP_NUM_THREADS` o `omp_set_num_threads`.

5. Validación

5.1. Referencia numpy

Este script genera un modelo *falso* con el contrato v0 y las salidas de referencia. Mientras no exista el modelo, es tu banco de pruebas; luego se cambia la parte aleatoria por la exportación real.

```
import numpy as np, pathlib
rng = np.random.default_rng(0)
N, d, h1 = 23335, 128, 256; d0 = 2*d + 3
out = pathlib.Path("c/datos_prueba"); out.mkdir(parents=True, exist_ok=True)
V = rng.standard_normal((N, d)).astype("<f4"); V /= np.linalg.norm(V, axis=1, keepdims=
    True)
W1 = (rng.standard_normal((h1, d0)) / np.sqrt(d0)).astype("<f4"); b1 = np.zeros(h1, "<f4
    ")
W2 = (rng.standard_normal((d, h1)) / np.sqrt(h1)).astype("<f4");  b2 = np.zeros(d, "<f4"
    )
A_idx = rng.choice(N, 60, replace=False).astype("<i4"); A_w = rng.uniform(.2, 1, 60).
    astype("<f4")
B_idx = rng.choice(N, 8, replace=False).astype("<i4");  B_w = np.ones(8, "<f4")
excl = np.union1d(A_idx, B_idx).astype("<i4")

def media(idx, w):
    if w.sum() <= 1e-9: return np.zeros(d), 1.0
    return (w[:, None].astype("f8") * V[idx]).sum(0) / w.sum(), 0.0
a, fA = media(A_idx, A_w); b, fB = media(B_idx, B_w)
x = np.concatenate([a, b, [fA, fB, np.log1p(len(A_idx) + len(B_idx))]]).astype("f4")
z = W2 @ np.maximum(0, W1 @ x + b1) + b2
u = z / max(np.linalg.norm(z), 1e-12)
```

```
s = V @ u; s[excl] = -np.inf
top = np.lexsort((np.arange(N), -s))[:200]          # score desc., empate -> índice
    menor

for nom, arr in dict(V=V, W1=W1, b1=b1, W2=W2, b2=b2, usuario_A_idx=A_idx, usuario_A_w=
    A_w,
                     usuario_B_idx=B_idx, usuario_B_w=B_w, usuario_excl=excl).items():
    arr.tofile(out / f"{nom}.{'i32' if arr.dtype.kind == 'i' else 'f32'}")
u.astype("<f4").tofile(out/"ref_u.f32"); s.astype("<f4").tofile(out/"ref_scores.f32")
top.astype("<i4").tofile(out/"ref_top.i32")
```

(`meta.txt` lo escribes a mano o con cuatro `print`.)

5.2. Qué comparar

Salida	Criterio
u	$\max_c \lvert u_c^C - u_c^{np}\rvert < 10^{-4}$
scores	$\max_i \lvert s_i^C - s_i^{np}\rvert < 10^{-4}$ sobre los no excluidos; los excluidos, $-\infty$ en los dos
top-200	mismo conjunto y mismo orden, salvo en empates: si difieren, los scores en esas posiciones deben coincidir dentro de 10^{-4}
MMR	con $\lambda = 1$, igual al top-20; con $\lambda = 0{,}7$, igual a una versión numpy de diez líneas

Por qué no da exactamente igual

Con OpenMP y SIMD el orden de las sumas cambia, y en float32 eso mueve el último decimal. Por eso tolerancia 10^{-4} y no igualdad. Si sale 10^{-2}, no es redondeo: es un error (casi siempre el orden de la entrada o W transpuesta).

5.3. Casos límite (un test cada uno)

1. **Bolsa B vacía**: $\bar{\mathbf{b}} = \mathbf{0}$, $f_B = 1$, sin NaN.
2. **Usuario sin historial** (A y B vacías): $\mathbf{x} = [\mathbf{0}, \mathbf{0}, 1, 1, 0]$; **u** sale de los sesgos; no debe haber NaN ni cuelgue (si $\mathbf{z} = \mathbf{0}$, `normalizar_l2` lo deja igual).
3. **k mayor que las películas no excluidas**: con $N = 300$ y 150 excluidas, `top_k` con $k = 200$ devuelve 150 y ningún $-\infty$; MMR con $nc < 20$ devuelve nc.
4. **Archivo con tamaño erróneo**: `cargar_modelo` devuelve error, no lee basura.
5. **Empates**: scores todos iguales $\Rightarrow$ top-k = los k índices menores.
6. **Hilos**: el resultado con 1 y con 12 hilos es el mismo top-200.

6. Benchmarks

6.1. Orden de magnitud antes de medir

$V\mathbf{u}$ son $23\,335 \times 128 \approx 3$ M multiplicaciones-suma y leer V entero: $23\,335 \cdot 128 \cdot 4 \approx 12$ MB (no cabe en la caché L3 de 12 MB con holgura, así que manda la memoria). Con AVX2 a ~GHz eso es del orden de **1 ms en un hilo**, y con varios hilos se acercará al límite del ancho de banda de memoria (~0,2–0,5 ms). El resto (medias, MLP, heap, MMR) debería sumar $< 0{,}2$ ms. Si mides 50 ms, algo no vectoriza o hay una copia de más.

6.2. Qué medir

El PC de Arnau es un i7-1355U: 10 núcleos (2 de rendimiento + 8 eficientes) y 12 hilos. Mide con 1000 usuarios reales (o simulados) y da p50 y p95 de la latencia *por usuario*:

Variante	p50 (ms)	p95 (ms)
Python + numpy (la referencia de arriba)		
C, 1 hilo		
C, OpenMP 2 / 4 / 8 / 12 hilos		
C, lote de usuarios en paralelo (rendimiento: usuarios/s)		

y el **desglose por etapa** (medias, MLP, scores, máscara, top-k, MMR) con `omp_get_wtime()` alrededor de cada una, acumulando en un array de tiempos. Repite cada medición varias veces y descarta las primeras (caché fría). Con núcleos híbridos, espera que de 2 a 4 hilos escale bien y de 8 a 12 mucho menos: es un resultado, no un fallo.

6.3. Llamar a la `.dll` desde Python

Lo más cómodo es exponer una función que reciba punteros a arrays numpy (sin que C lea archivos del usuario):

```
MOTOR_API int  motor_cargar(const char *carpeta);   /* guarda el Modelo en una variable
    global */
MOTOR_API int  motor_recomendar(const int32_t *A_idx, const float *A_w, int nA,
                                const int32_t *B_idx, const float *B_w, int nB,
                                const int32_t *excl, int nE, int k_cand, int k_final,
                                float lambda, int32_t *out, double *tiempos);
```

```
import ctypes, os, numpy as np
from numpy.ctypeslib import ndpointer
os.add_dll_directory(r"C:\Program Files (x86)\Simply Fortran 3\mingw-w64\bin")  #
    libgomp
lib = ctypes.CDLL(os.path.abspath("c/build/motor.dll"))
i32 = ndpointer(np.int32, flags="C_CONTIGUOUS"); f32 = ndpointer(np.float32, flags="
    C_CONTIGUOUS")
lib.motor_cargar.argtypes = [ctypes.c_char_p]
lib.motor_recomendar.argtypes = [i32, f32, ctypes.c_int, i32, f32, ctypes.c_int, i32,
    ctypes.c_int,
    ctypes.c_int, ctypes.c_int, ctypes.c_float, i32, ndpointer(np.float64, flags="
    C_CONTIGUOUS")]
lib.motor_recomendar.restype = ctypes.c_int
assert lib.motor_cargar(b"c/datos_prueba") == 0
out = np.empty(20, np.int32); t = np.zeros(6)
n = lib.motor_recomendar(A_idx, A_w, len(A_idx), B_idx, B_w, len(B_idx), excl, len(excl)
    ,
                         200, 20, 0.7, out, t)
```

ctypes

Si un array no es `int32`/`float32` contiguo, `ndpointer` lanza un error (bien); sin `argtypes`, ctypes pasaría basura sin avisar. Para medir la latencia *del motor* usa los tiempos que devuelve C; el tiempo medido desde Python incluye unos microsegundos de `ctypes`, que también conviene reportar.

6.4. Comparación con `hnswlib`

En Python, sobre la misma V: `hnswlib.Index(space="ip", dim=128)` (producto interno; con V normalizada es el coseno), `init_index(max_elements=N, M=16, ef_construction=200)`, `add_items(V)` y `knn_query(u, k=200)` con `ef` $\in \{200, 400, 800\}$. Mide:

- latencia p50/p95 por consulta (y con `set_num_threads(1)` para comparar con C de 1 hilo);
- **recall@200 del ANN**: $|\text{HNSW}_{200} \cap \text{exacto}_{200}|/200$, frente al top-200 exacto (sin máscara en los dos, porque `hnswlib` no la aplica; o pidiendo $200 + n_E$ y filtrando);
- tiempo de construcción del índice y memoria.

La conclusión esperada (H9): con $N = 23\,335$ HNSW es algo más rápido pero pierde recall, y el exacto ya cumple los 5 ms. HNSW compensaría con catálogos de millones.

6.5. Para la memoria (E2/E3)

Una gráfica de latencia frente a hilos (con la línea de 5 ms), una tabla del desglose por etapa y la curva recall@200 frente a latencia de HNSW con el punto del exacto (recall 1).

7. Estructura y orden de trabajo

7.1. Carpetas

```
c/
  include/motor.h        # structs, firmas, MOTOR_API
  src/io.c               # leer_f32, leer_i32, leer_meta, aligned_alloc_64
  src/modelo.c           # cargar_modelo, liberar_modelo
  src/usuario.c          # media_bolsa, matvec, mlp_usuario, normalizar_l2
  src/busqueda.c         # scores, aplicar_mascara, top_k (+ versión OpenMP)
  src/mmr.c
  src/api.c              # motor_cargar, motor_recomendar (para ctypes)
  tests/test_motor.c     # compara con ref_*.f32 y casos límite
  python/generar_prueba.py  python/bench.py  python/bench_hnsw.py
  build.bat              # las dos líneas de gcc
  datos_prueba/          # generado, fuera de git
  build/                 # fuera de git
```

7.2. Pasos (cada uno se comprueba antes de seguir)

1. `generar_prueba.py` + `io.c`: leer `V.f32` y que la suma de la fila 0 coincida con numpy.
2. `cargar_modelo` con `meta.txt`; test de archivo de tamaño erróneo.
3. `media_bolsa`: compara $\bar{\mathbf{a}}$ con numpy; test de bolsa vacía.
4. `mlp_usuario` + `normalizar_l2`: $\mathbf{u}$ dentro de 10^{-4}.
5. `scores` secuencial + máscara: scores dentro de 10^{-4}.
6. `top_k` con heap: mismo top-200; tests de empates y de $k >$ disponibles.
7. `mmr`: $\lambda = 1$ da el top-20; ejemplo de tres películas.
8. OpenMP en `scores` y top-k por hilos: mismo resultado con 1 y 12 hilos.
9. `api.c` + `.dll` + `ctypes`: mismo resultado que el ejecutable.

10. `bench.py` (numpy vs C, hilos, desglose) y `bench_hnsw.py`.

11. Cuando exista el modelo: cambiar la parte aleatoria por la exportación real y cerrar el contrato v1.

Lo que queda abierto

Del contrato son provisionales: h_1, las banderas exactas y el n del logaritmo, `relu_final`, el orden de la entrada y los nombres de archivo. Lo estable: float32/int32/int64 little-endian row-major, V normalizada $[N \times d]$, $d = 128$, la máscara C14/F8 y la salida (200 candidatos $\rightarrow$ 20 por MMR).